



# Function Returns Explained

## Brief Overview

This note covers **functions with return** and was created from an [uploaded document](#). It covers function syntax, **return usage**, nested functions, **example calculations**, and practical usage scenarios.

## Key Points

- Define functions with the function keyword.
  - Use return to output values.
  - Nest functions for modularity.
  - Apply examples to real-world change calculations.
- 

## Functions with Return

**Definition:** A function that processes input values **and** provides an output using the return keyword.

- The value after return becomes the result of the function call.
- The result can be stored in a variable, passed to another function, or printed.
- If no return statement is present, the function runs but yields **no value**.

## Key Components

- **function keyword** – starts the function definition.
- **Function name** – follows the function keyword (e.g., getMilk).
- **Parameters** – placed inside parentheses; hold input values.
- **Body** – code inside curly braces { ... }.
- **return statement** – placed before the closing brace to output a value.

## Example Flow

1. Call getMilk(4).
2. Inside the function, compute money % 1.5 (remainder after buying bottles).
3. return the remainder (1).
4. The caller assigns the result: let change = getMilk(4); → change equals 1.

## Practical Example

## Using return to Get Change

| Input (money) | Calculation (money % 1.5)                     | Returned Change |
|---------------|-----------------------------------------------|-----------------|
| \$3           | $\$3 \div 1.5 \rightarrow$ remainder <b>0</b> | <b>0</b>        |
| \$4           | $\$4 \div 1.5 \rightarrow$ remainder <b>1</b> | <b>1</b>        |

- The returned change can be logged: `console.log(change)`.

## Functions Inside Functions

| Function                                               | Parameters                    | Returned Value                                                               | Purpose                                  |
|--------------------------------------------------------|-------------------------------|------------------------------------------------------------------------------|------------------------------------------|
| <code>calcBottles(startingMoney, costPerBottle)</code> | Money amount, cost per bottle | <code>startingMoney / costPerBottle</code>                                   | Computes how many bottles can be bought. |
| <code>calcChange(startingAmount, costPerBottle)</code> | Money amount, cost per bottle | <code>startingAmount % costPerBottle</code>                                  | Computes remaining change.               |
| <code>getMilk(money, costPerBottle)</code>             | Money amount, cost per bottle | Calls <code>calcBottles</code> and <code>calcChange</code> , returns change. | Coordinates the whole purchase process.  |

## Combined Usage (JavaScript)

```
function calcBottles(startingMoney, costPerBottle) {
  return startingMoney / costPerBottle;
}

function calcChange(startingAmount, costPerBottle) {
  return startingAmount % costPerBottle;
}

function getMilk(money, costPerBottle) {
  const bottles = calcBottles(money, costPerBottle);
  const change = calcChange(money, costPerBottle);
  console.log(`Bought ${bottles} bottles, change: $${change}`);
  return change;
}

// Example call
let change = getMilk(5, 1.5); // Logs: Bought 3.333... bottles, change: $0.5
```

- getMilk **returns** the change, which can be stored (let change = ...) or used directly in other expressions.

## Function Syntax Summary

- function ( , , ...) { ... return ; }
- **Parameters** receive input values; **return** sends output back.
- The calling line receives the returned value on the right-hand side of the assignment.

## Advantages of Return-Enabled Functions

- **Reusability:** Same logic can be invoked with different inputs.
  - **Modularity:** Separate calculations (e.g., bottles vs. change) into distinct functions.
  - **Maintainability:** Changing the cost per bottle only requires updating the argument, not the internal math.
- 

### Quick Reference Checklist

- [ ] Define function with function keyword.
- [ ] List needed parameters inside parentheses.
- [ ] Perform calculations inside the curly braces.
- [ ] End with return ; to output a value.
- [ ] Capture the output when calling the function (let result = funcName(args);).